로우 레벨 디자인 — DirectX12 TPS 4 인 개인전

이 문서는 전체 순서도에 그려진 일 하나하나를 '어느 클래스의 어느 함수가 하는지' 정한 설계서다.

- **장 구성:** 0 장 용어 풀이, 1 장 서버와 클라가 주고받는 데이터(패킷)와 보내는 규칙, 2~5 장 클래스와 함수, 6 장 순서도의 상자마다 그 일을 하는 함수, 7 장 스레드와 임계 영역, 8 장 구현 규칙, 9~10 장 부록.
- **함수 표 읽는 법:** 한 줄이 함수 하나다. 리턴 타입 + 함수명 + (인자)를 이어 쓰면 C++ 선언이 된다. 예: uint8_t + TryRegisterSeat + (SOCKET sock, ERejectReason& outReason) -> uint8_t TryRegisterSeat(SOCKET sock, ERejectReason& outReason);
- **인자 이름이 out 으로 시작하면** (outReason, outEvents 등) 함수가 결과를 그 변수에 채워서 돌려준다는 뜻이다.
- **표시:** (새) = 이번에 새로 만드는 것. (기존) = 베이스 게임에 이미 있는 것. (기존, 바뀜) = 베이스 게임에 있던 함수에 하는 일이 더해지는 것. (제안) = 다른 문서에 없고 이 문서에서 처음 정한 방법이라 팀이 바꿀 수 있는 것. 다른 자료에서 가져온 내용은 끝에 (출처: 자료 이름)을 적었다. 출처는 패킷 정의서, 전체 순서도, 베이스 게임 코드 세 가지다.

0. 용어 풀이

용어	뜻
큐(이벤트 큐)	받은 TCP 패킷을 도착 순서대로 쌓아 두는 목록. 클라의 TCP 수신 스레드가 넣고, 메인 스레드가 꺼낸다.
자리	서버가 접속한 클라를 적어 두는 칸. 4 칸이 있고, 칸 번호(0~3)가 그 플레이어의 PlayerId 가 된다.
PlayerId	플레이어 번호 0~3. 255(INVALID_PLAYER_ID)는 '없음'이라는 뜻이다(거절됨, 승자 없음).
World	서버 안의 게임 세계. 기체, 적, 총알과 이들의 이동, 피격 계산.
유령(Ghost)	죽은 플레이어의 상태. 화면을 보며 움직일 수만 있고 쏘거나 맞지 않는다. 다른 플레이어 화면에는 보이지 않는다.
씬(화면)	클라의 화면 하나. 시작 화면, 로비 화면, 게임 화면, 결과 화면 4 개가 있다.
로비	게임 시작 전에 모여서 준비 버튼을 누르는 화면, 또는 그 단계.
일련번호(sequence / tick)	패킷에 붙이는 번호. 보낼 때마다 1 씩 커지므로, 받은 번호가 마지막으로 받은 번호보다 작거나 같으면 늦게 도착한 옛날 패킷이라 버린다. 입력 패킷은 sequence, 상태 패킷은 tick 을 쓴다.

1. 공통 전제와 프로토콜

프로토콜은 서버와 클라가 주고받는 데이터의 모양과 순서에 대한 약속이다. 한 번만 보내고 반드시 도착해야 하는 정보는 TCP 로, 1/30 초마다 새로 보내서 하나쯤 빠져도 되는 정보는 UDP 로 보낸다.

패킷은 클라가 보내는 것 2 종, 서버가 보내는 것 8 종이고, 모두 1-5 의 구조체를 그대로 보낸다(전체 코드는 부록 Protocol.h).

1-1. 공통 전제

항목	정한 값	출처
주기	서버 틱과 클라 프레임 모두 1 초에 30 번(1/30 초 = 약 33.3ms 마다). 서버는 틱마다 게임 세계를 1/30 초만큼 진행한다(시간 값 dt = 1/30 초로 고정).	패킷 정의서, 전체 순서도
인원	최대 4 명. 2 명 이상 모이면 시작할 수 있다. PlayerId 는 0~3, 255 = 없음.	패킷 정의서
포트와 IP	서버는 TCP 9000 번, UDP 9001 번 포트에서 기다린다(번호는 예시). 클라의 UDP 포트는 정하지 않고 운영체제가 빈 번호를 준다. 서버는 클라가 보낸 입력 패킷에서 그 클라의 주소를 알아내 그 주소로 답한다. 서버 IP 는 클라 코드의 전역 상수 SERVER_IP 에 적는다.	패킷 정의서(포트), 9 장 부록 1
동기화 도구	여러 스레드가 같이 쓰는 데이터는 임계 영역(잠금)으로 지킨다. 잠금은 '동시에 만지지 않게'만 해 줄 뿐 '누가 먼저'는 정하지 못하므로, 스레드 사이의 순서(모두 준비된 다음에 게임 시작)는 이벤트로 맞춘다.	전체 순서도

1-2. 어느 통로로 보내나

통로	보내는 것	보내는 방법	이 통로를 쓰는 이유
TCP (양쪽 방향)	접속 결과, 준비, 로비 상태, 게임 시작, 사망, 게임 결과	클라마다 TCP 연결 1 개. 클라가 접속하면 서버가 바로 접속 결과를 보낸다(접속을 요청하는 패킷은 따로 없다).	한 번만 보내는 정보라서 하나라도 빠지거나 순서가 바뀌면 화면이 서버와 달라진다. TCP 는 빠짐없이, 보낸 순서대로 도착시켜 준다.
UDP 클라 -> 서버	키 입력. 매 프레임	서버의 UDP 주소(9001 번 포트)로 보낸다(sendto).	'지금 무엇을 누르고 있나'라서 가장 최근 것 하나만 있으면 된다. 빠진 것을 다시 받으려고 기다리면 조작이 늦어진다.
UDP 서버 -> 클라	상태 3 종(플레이어, 적, 총알). 매 틱	클라마다 따로 보낸다(유니캐스트 = 한 명에게만 보내기). 받을 주소는 그 클라의 입력 패킷을 받을 때(recvfrom) 알아 둔 주소다.	1/30 초마다 새 값이 오므로, 늦게 온 것은 버리고 다음 것을 쓰면 된다.

모든 데이터를 한 패킷에 몰아 보내지 않는다. 상태는 플레이어, 적, 총알 3 개 패킷으로 나누고, 살아 있는 적과 날아가는 총알만 담는다. 사망처럼 한 번 일어나는 일은 상태 패킷에 넣지 않고 TCP 의 SC_Event 로 따로 보낸다.

1-3. 헤더와 보내기, 받기 규칙

항목	규칙	출처
헤더	모든 패킷은 3 바이트 헤더(PacketHeader)로 시작한다. 앞 2 바이트 size = 헤더를 포함한 패킷 전체 크기(바이트), 뒤 1 바이트 type = 패킷 종류 번호(1-4 표의 괄호 숫자).	패킷 정의서
묶는 방법	패킷 구조체를 #pragma pack(push, 1)로 선언해 값 사이에 빈 바이트가 끼지 않게 하고, 구조체를 그대로 send / sendto 로 보낸다. 자료형은 크기가 정해진 uint8_t(1 바이트), int8_t(1 바이트), uint16_t(2 바이트), uint32_t(4 바이트), float(4 바이트)만 쓴다.	패킷 정의서
바이트 순서	여러 바이트로 된 숫자를 어떤 순서로 적는지의 규칙이다. IP 와 포트만 네트워크 표준 순서로 바꾸고(htons 등), 게임 데이터는 바꾸지 않고 보낸다. 참가 PC 가 모두 x64 Windows 라서 순서가 같기 때문이다.	패킷 정의서
TCP 보내기	서버의 TCP 전송은 모두 SendPacket() 함수 하나로 한다. 이 함수는 그 클라의 송신 잠금을 잡은 채, 패킷이 다 나갈 때까지 send()를 반복한다(send 한 번에 다 안 나갈 수 있기 때문).	패킷 정의서
TCP 받기	TCP 는 패킷 경계 없이 이어진 데이터로 오므로 두 번에 나눠 받는다. 먼저 헤더 3 바이트를 다 받고, 헤더의 size 에서 3 을 뺀 나머지를 마저 받는다. 둘 다 recv(..., MSG_WAITALL)로 받는다(요청한 바이트가 다 올 때까지 기다리는 옵션).	패킷 정의서
UDP 받기	UDP 는 recvfrom() 한 번에 패킷 하나가 통째로 온다. 받은 바이트 수가 헤더의 size 와 다르면 깨진 패킷으로 보고 버린다.	패킷 정의서
개수가 변하는 패킷	상태 3 종은 배열을 최대 크기로 선언해 두고, 보낼 때는 앞부분 8 바이트(헤더 3 + tick 4 + count 1)와 실제 개수(count)만큼의 칸만 보낸다. 크기 = 8 + count × 한 칸 크기.	패킷 정의서
일련번호	입력의 sequence 와 상태의 tick 은 1 씩 커지고, 게임이 새로 시작돼도 0 으로 되돌리지 않는다. 받은 번호가 마지막으로 받은 번호보다 작거나 같으면 늦게 도착한 옛날 패킷이므로 버린다.	패킷 정의서

1-4. 패킷 목록

패킷 이름 옆 괄호 안 숫자는 **패킷 종류 번호**다. 헤더의 마지막 1 바이트(type)에 이 값이 들어가고, 받는 쪽은 이 값을 보고 나머지를 어떤 구조체로 읽을지 정한다. 클라 -> 서버는 1 부터, 서버 -> 클라는 100 부터 매겨서 번호만 봐도 보낸 쪽을 알 수 있다(출처: 패킷 정의서의 EPacketType).

패킷 (type 값)	방향, 통로	보내는 때	크기(바이트)	보내는 함수 (스레드)	받아서 처리하는 함수 (스레드)
CS_Ready (1)	클라 -> 서버, TCP	로비에서 준비 버튼을 처음 누를 때 1 번	3	CNetworkClient::SendReady (클라 메인)	CGameServer::TcpCommLoop 가 받아 SetReady (서버 TCP 통신)
CS_Input (2)	클라 -> 서버, UDP	게임 중 매 프레임	16	CNetworkClient::SendInput (클라 메인)	CGameServer::UdpRecvLoop (서버 UDP 수신)

패킷 (type 값)	방향, 통로	보내는 때	크기(바이트)	보내는 함수 (스레드)	받아서 처리하는 함수 (스레드)
SC_ConnectResult (100)	서버 -> 접속한 그 클라, TCP	클라가 접속하자마자 1 번	6	CGameServer::SendConnectResult (서버 TCP 통신)	CNetworkClient::TcpRecvLoop 가 큐에 넣음 (클라 TCP 수신) -> 메인이 꺼내 CStartScene::OnNetEvent (클라 메인)
SC_LobbyState (101)	서버 -> 전원, TCP	누가 접속했을 때, 누가 준비했을 때, 게임이 끝나 로비로 돌아갈 때	12	CGameServer::BroadcastLobbyState (서버 TCP 통신, 서버 게임 루프)	큐 -> CGameFramework::ProcessNetEvents 가 저장 -> CMapSelectScene::SetLobbyState (클라 메인)
SC_GameStart (102)	서버 -> 전원, TCP	게임을 준비할 때 1 번	61	CGameLoop::PrepareGame (서버 게임 루프)	큐 -> ProcessNetEvents -> ChangeScene 으로 게임 화면(CGameScene) 만들기 (클라 메인)
SC_PlayerStates (103)	서버 -> 클라마다, UDP	매 틱	8 + 20 × 인원 (최대 88)	CGameLoop::SendStates (서버 게임 루프)	CNetworkClient::UdpRecvLoop 가 저장 (클라 UDP 수신) -> CGameScene::ApplyStates (클라 메인)
SC_EnemyStates (104)	서버 -> 클라마다, UDP	매 틱	8 + 17 × 살아 있는 적 수 (최대 552)	CGameLoop::SendStates (서버 게임 루프)	UdpRecvLoop -> ApplyStates
SC_BulletStates (105)	서버 -> 클라마다, UDP	매 틱	8 + 12 × 날아가는 총알 수 (최대 776)	CGameLoop::SendStates (서버 게임 루프)	UdpRecvLoop -> ApplyStates
SC_Event (106)	서버 -> 전원, TCP	누가(무엇이) 죽은 틱에	17	CGameLoop::SendEvents (서버 게임 루프)	큐 -> CGameScene::OnNetEvent (클라 메인)
SC_GameResult (107)	서버 -> 전원, TCP	살아 있는 플레이어가 1 명 이하가 된 틱에 1 번	4	CGameLoop::SendResult (서버 게임 루프)	큐 -> CGameScene::OnNetEvent -> 결과 화면(CEndScene) (클라 메인)

전송량: 가장 많을 때(Map1 의 적 15 명이 모두 살아 있고 총알 64 발이 모두 날아갈 때) 서버가 클라 1 명에게 한 틱에 보내는 양은 88 + 263 + 776 = 1,127 바이트다. 1 초에 30 틱이므로 클라 1 명당 약 34KB/초, 4 명이면 서버가 보내는 양은 약 135KB/초다(출처: 패킷 정의서).

1-5. 패킷 구조체

헤더(3 바이트) 뒤에 오는 값을 순서대로 적었다. 괄호 안은 자료형과 뜻이다.

구조체	헤더 뒤에 오는 값	크기(바이트)
PacketHeader	size(uint16_t, 패킷 전체 크기), type(uint8_t, 패킷 종류 번호)	3
CS_Ready	없음(헤더만)	3
CS_Input	playerId(uint8_t, 보낸 플레이어 번호), sequence(uint32_t, 입력 일련번호), moveForward(int8_t, 앞뒤 이동 -1=S, 0, 1=W), moveRight(int8_t, 좌우 이동 -1=A, 0, 1=D), cameraYaw(float, 카메라 좌우 방향, 도), fire(uint8_t, 좌클릭 중이면 1), shieldPressCount(uint8_t, E 를 누른 누적 횟수)	16
SC_ConnectResult	accepted(uint8_t, 1 = 승인, 0 = 거절), playerId(uint8_t, 받은 번호, 거절이면 255), rejectReason(uint8_t, 거절 이유 = ERejectReason)	6
SC_LobbyState	playerCount(uint8_t, 접속 인원), players[4] = 칸마다 { playerId(uint8_t), ready(uint8_t, 준비했으면 1) }	12
SC_GameStart	mapNumber(uint8_t, 맵 1~3), playerCount(uint8_t, 참가 인원), players[4] = 칸마다 { playerId(uint8_t), playerType(uint8_t, 기체 종류 = EPlayerType), x, y, z(float, 시작 위치) }	61
SC_PlayerStates	tick(uint32_t, 틱 번호), count(uint8_t, 담긴 플레이어 수), players[count] = 칸마다 { playerId, state(살아 있음/유령 = EPlayerState), shieldEnabled(쉴드 켜짐 1), health(체력 0~100) — 모두 uint8_t, x, y, z(위치), yaw(바라보는 방향, 도) — 모두 float }	8 + 20 × count
SC_EnemyStates	tick(uint32_t), count(uint8_t, 살아 있는 적 수), enemies[count] = 칸마다 { enemyIndex(uint8_t, 적 번호), x, y, z, yaw(float) }	8 + 17 × count
SC_BulletStates	tick(uint32_t), count(uint8_t, 날아가는 총알 수), bullets[count] = 칸마다 { x, y, z(float, 위치) }	8 + 12 × count
SC_Event	eventType(uint8_t, 플레이어 사망/적 사망 = EEventType), subjectId(uint8_t, 죽은 플레이어의 PlayerId 또는 죽은 적의 enemyIndex), x, y, z(float, 죽은 위치)	17
SC_GameResult	winnerPlayerId(uint8_t, 승자 번호. 승자가 없으면 255 = 모두 패배)	4

열거형(enum)은 5 개다. EPacketType(패킷 종류 번호), EPlayerType { None, Helicopter, Tank }(기체 종류), EPlayerState { Alive = 살아 있음, Ghost = 유령 }, EEventType { PlayerDied = 플레이어 사망, EnemyDied = 적 사망 }, ERejectReason { None = 거절 아님, ServerFull = 4 자리가 다 참, AlreadyStarted = 이미 게임 중 }. 지금이 로비인지 게임 중인지는 서버 안에만 두고 패킷으로 보내지 않는다.

2. 클래스 한눈에 보기

서버는 네트워크를 맡는 CGameServer 와 게임 진행을 맡는 CGameLoop 가 중심이다. 클라는 CGameFramework 가 네트워크를 맡는 CNetworkClient 를 하나 가지고, 화면(씬) 4 개가 필요할 때 그것을 쓴다.

프로그램	클래스	하는 일	함수가 도는 스레드	가지고 있는 것
서버	CGameServer (새)	소켓 준비, 접속 받기, 자리 등록, 준비, 시작 확인, TCP 보내기, UDP 입력 받기	서버 메인, 서버 TCP 통신(클라 1 명당 1 개), 서버 UDP 수신	자리 4 칸(CClientSession), 입력 4 칸(InputSlot), CGameLoop
서버	CClientSession (새, 구조체)	자리 한 칸. 접속한 클라 1 명의 정보	(데이터라 스레드 없음)	TCP 소켓, 준비 여부, 송신 잠금
서버	InputSlot (새, 구조체)	입력 한 칸. 플레이어 1 명의 마지막 입력과 UDP 주소	(데이터라 스레드 없음)	마지막 입력, 입력 일련번호, UDP 주소
서버	CGameLoop (새)	게임 한 판 준비 -> 1/30 초마다 진행 -> 결과 보내기 -> 로비로 되돌리기	서버 게임 루프	CGameWorld, CTerrainGameMap, 틱 번호
서버	CGameWorld 등 (기존)	이동, 적 AI, 총알, 피격 계산	서버 게임 루프	기체, 적, 총알
클라	CNetworkClient (새)	접속, 패킷 보내기, TCP, UDP 받기, 이벤트 큐와 최신 상태 보관	클라 메인, 클라 TCP 수신, 클라 UDP 수신	소켓 2 개, 이벤트 큐, 최신 상태
클라	CGameFramework (기존)	매 프레임 진행, 화면 바꾸기, 받은 패킷을 화면에 나눠 주기	클라 메인	CNetworkClient, 지금 화면, 마지막 로비 상태
클라	CSceneBase (기존)	화면 4 개의 부모 클래스	클라 메인	네트워크 객체 포인터
클라	CStartScene (기존)	시작 화면. 제목을 클릭하면 접속	클라 메인	—
클라	CMapSelectScene (기존)	로비 화면. 준비 버튼과 자리 4 칸	클라 메인	로비 상태, 준비를 보냈는지
클라	CGameScene (기존)	게임 화면. 입력 보내기, 받은 상태 보여 주기	클라 메인	화면에 그릴 기체, 적, 총알, PlayerId 별 기체 표
클라	CEndScene (기존)	결과 화면. 승리 또는 패배	클라 메인	—

3. 서버 클래스

3-1. CGameServer (새)

서버 프로그램 전체를 대표하는 클래스다. 소켓, 자리 4 칸, 입력 4 칸, 잠금(임계 영역)을 가지고 있고, 서버 메인, 서버 TCP 통신, 서버 UDP 수신 스레드가 이 클래스의 함수를 실행한다.

멤버 변수

형식	이름	뜻
SOCKET	m_listenSock	클라의 접속을 기다리는 TCP 소켓(포트 9000)
SOCKET	m_udpSock	UDP 소켓(포트 9001). UDP 수신 스레드가 입력을 받을 때, 게임 루프 스레드가 상태를 보낼 때 같이 쓴다
CClientSession	m_seats[MAX_PLAYERS]	자리 4 칸(3-2). 칸 번호 0~3 이 그 플레이어의 PlayerId 다
bool	m_inGame	지금 게임 중이면 true, 로비(게임 전 대기)면 false
CRITICAL_SECTION	m_seatLock	자리 잠금. m_seats 의 used, ready 와 m_inGame 을 한 번에 한 스레드만 만지게 한다
InputSlot	m_inputs[MAX_PLAYERS]	입력 4 칸(3-3). 플레이어별 마지막 입력과 UDP 주소
CRITICAL_SECTION	m_inputLock	입력 잠금. m_inputs 를 한 번에 한 스레드만 만지게 한다
HANDLE	m_startEvent	게임 시작 신호(이벤트). 자동 리셋 방식이라 게임 루프가 신호를 한 번 받으면 저절로 다시 '신호 없음'이 된다
CGameLoop	m_gameLoop	게임 진행을 맡는 객체(3-4)
HANDLE	m_udpThread, m_gameLoopThread	UDP 수신 스레드와 게임 루프 스레드의 핸들(만든 스레드를 가리키는 값. 서버를 끌 때 스레드가 끝나길 기다리는 데 쓴다)

함수

리턴 타입	함수명	인자	역할	스레드
bool	Start	없음	서버를 켤 때 한 번 부른다. 모두 성공하면 true, 하나라도 실패하면 false 를 돌려준다. 하는 순서: ① 윈속 시작(WSAStartup) ② TCP 소켓을 9000 번 포트에 열고 접속을 기다리게 한다(bind, listen) ③ UDP 소켓을 9001 번 포트에 연다(bind) ④ 잠금 2 개와 시작 이벤트를 만든다 ⑤ UDP 수신 스레드와 게임 루프 스레드를 시작한다.	서버 메인

리턴 타입	함수명	인자	역할	스레드
void	AcceptLoop	없음	클라의 접속을 기다리다가(accept) 접속이 오면, 그 클라를 맡을 TCP 통신 스레드를 하나 만들고 다시 기다린다. 이것을 계속 반복한다.	서버 메인
static DWORD WINAPI	TcpCommThread	LPVOID param: 서버 포인터와 그 클라의 소켓을 담은 값	TCP 통신 스레드가 시작되면 가장 먼저 실행되는 함수다(CreateThread 가 이 모양을 요구해서 static 이다). param 에서 서버 포인터와 소켓을 꺼내 TcpCommLoop 를 부른다.	서버 TCP 통신
void	TcpCommLoop	SOCKET sock: 이 클라와 연결된 TCP 소켓	클라 1 명을 맡는 스레드가 하는 일 전체다. ① TryRegisterSeat 로 자리 등록 ② SendConnectResult 로 접속 결과 보내기 ③ 승인이면 BroadcastLobbyState 로 전원에게 로비 상태 보내기 ④ 그 뒤로는 이 클라가 보내는 패킷(CS_Ready)을 기다렸다가 처리하기를 반복한다.	서버 TCP 통신
uint8_t	TryRegisterSeat	SOCKET sock: 이 클라의 TCP 소켓; ERejectReason& outReason: 거절 이유를 담아 돌려받을 변수	지금이 로비이고 빈 자리가 있으면, 가장 앞의 빈 자리에 이 클라를 적고 그 자리 번호(0~3 = PlayerId)를 돌려준다. 등록할 수 없으면 255(INVALID_PLAYER_ID)를 돌려주고 outReason 에 이유를 적는다(ServerFull = 4 자리가 다 참, AlreadyStarted = 이미 게임 중).	서버 TCP 통신
void	SendConnectResult	SOCKET sock: 이 클라의 TCP 소켓; uint8_t playerId: 받은 자리 번호. 거절이면 255; ERejectReason reason: 거절 이유. 승인이면 None	이 클라에게 접속 결과(SC_ConnectResult)를 보낸다. 거절이면 보낸 뒤 소켓을 닫는다.	서버 TCP 통신
void	SetReady	uint8_t playerId: 준비 버튼을 누른 플레이어 번호	그 자리를 '준비 완료'(ready = true)로 바꾼다.	서버 TCP 통신

리턴 타입	함수명	인자	역할	스레드
bool	TryStartGame	없음	세 조건을 확인한다: 로비 중인가, 접속한 사람이 2 명 이상인가, 접속한 사람이 모두 준비했나. 모두 맞으면 m_inGame 을 true 로 바꾸고 true 를 돌려준다. 확인과 바꾸기를 한 번의 잠금 안에서 하므로, 두 클라가 동시에 준비해도 true 는 한 스레드만 받는다. true 를 받은 스레드만 SetEvent 로 게임 루프를 깨운다.	서버 TCP 통신
void	UdpRecvLoop	없음	UDP 로 오는 입력 패킷(CS_Input)을 받기(recvfrom)를 반복한다. 크기, 플레이어 번호, 일련번호를 검사해서 통과한 것만 그 플레이어 칸에 덮어쓰고, 보낸 쪽 주소도 함께 저장한다.	서버 UDP 수신
bool	SendPacket	CClientSession& s: 보낼 상대의 자리; const char* data: 보낼 패킷; int len: 보낼 바이트 수	그 자리의 송신 잠금(sendLock)을 잡은 채 패킷이 다 나갈 때까지 보낸다. 서버의 TCP 전송은 모두 이 함수를 거친다. 실패하면 false 를 돌려준다.	모든 서버 스레드
void	SendToAll	const char* data: 보낼 패킷; int len: 보낼 바이트 수	접속한 모든 자리에 SendPacket 을 부른다.	서버 TCP 통신, 서버 게임 루프
void	BroadcastLobbyState	없음	자리 목록을 보고 로비 상태(SC_LobbyState: 누가 접속했고 누가 준비했나)를 만들어 접속한 전원에게 보낸다.	서버 TCP 통신, 서버 게임 루프
void	GetParticipants	bool outJoined[MAX_PLAYERS]: 결과를 채울 배열	사람이 있는 자리는 true, 빈 자리는 false 로 채워 준다.	서버 게임 루프
void	CopyInputs	CS_Input outInputs[MAX_PLAYERS]: 결과를 채울 배열	4 명의 마지막 입력을 입력 잠금 안에서 복사해 준다.	서버 게임 루프
void	CopyUdpAddrs	sockaddr_in outAddrs[MAX_PLAYERS]: UDP 주소를 채울 배열; bool outHasAddr[MAX_PLAYERS]: 주소를 알고 있는지를 채울 배열	4 명의 UDP 주소와, 그 주소를 이미 알고 있는지(첫 입력을 받았는지)를 복사해 준다.	서버 게임 루프

리턴 타입	함수명	인자	역할	스레드
void	ClearInputs	없음	남아 있는 마지막 입력의 이동, 발사 값을 0 으로 지운다. 지난 판의 입력 때문에 새 판에서 기체가 혼자 움직이지 않게 하려는 것이다. 일련번호, 주소, E 누른 횟수는 그대로 둔다.	서버 게임 루프
void	ResetLobby	없음	m_inGame 을 false 로 바꾸고 모든 자리의 준비를 푼다.	서버 게임 루프

3-2. CClientSession (새, 구조체) — 자리 한 칸

접속한 클라 1 명의 정보다. CGameServer 가 4 칸(m_seats[4])을 가지고 있다.

멤버 변수

형식	이름	뜻
bool	used	이 자리에 클라가 있으면 true
bool	ready	이 클라가 준비 버튼을 눌렀으면 true
SOCKET	sock	이 클라와 연결된 TCP 소켓
CRITICAL_SECTION	sendLock	이 자리의 송신 잠금. 두 스레드가 같은 소켓에 동시에 보내서 패킷이 섞이는 것을 막는다
HANDLE	thread	이 클라를 맡은 TCP 통신 스레드의 핸들

3-3. InputSlot (새, 구조체) — 입력 한 칸

플레이어 1 명이 마지막으로 보낸 입력과 그 플레이어의 UDP 주소다. CGameServer 가 4 칸(m_inputs[4])을 가지고 있다.

멤버 변수

형식	이름	뜻
bool	active	이 칸에 플레이어가 있으면 true. false 인 칸으로 온 입력은 버린다
uint32_t	lastSequence	마지막으로 받은 입력의 일련번호. 새로 온 번호가 이 값보다 작거나 같으면 늦게 도착한 옛날 입력이라 버린다. 자리 등록 때 0 으로 되돌린다
CS_Input	input	마지막으로 받은 입력
sockaddr_in	addr	이 플레이어의 UDP 주소(IP + 포트). 입력을 받을 때 recvfrom 이 알려 준다
bool	hasAddr	addr 를 한 번이라도 받았으면 true. false 면 아직 상태를 보낼 주소를 모른다

3-4. CGameLoop (새)

서버 게임 루프 스레드 하나가 쓰는 클래스다. 게임 한 판을 준비하고, 1/30 초마다 게임을 진행하고, 끝나면 로비로 되돌린다. 아래 함수는 모두 서버 게임 루프 스레드에서 실행된다.

멤버 변수

형식	이름	뜻
CGameServer*	m_server	서버 포인터. 자리, 입력을 읽고 패킷을 보내는 서버 함수를 쓰기 위해 가지고 있다
CTerrainGameMap*	m_map	이번 판의 맵(지형 높이, 적이 나타나는 위치)
CGameWorld	m_world	게임 세계(기체, 적, 총알과 그 계산). 게임 루프 스레드만 만지므로 잠금이 필요 없다
CGameTimer	m_timer	1/30 초 간격을 맞추는 타이머
CPlayer*	m_playerById[MAX_PLAYERS]	PlayerId 로 그 플레이어의 기체를 찾는 표. 빈 자리는 nullptr
uint8_t	m_lastShieldCount[MAX_PLAYERS]	지난 틱에 받은 'E 누른 누적 횟수'. 이번 값과 다르면 E 를 새로 누른 것이므로 쉴드를 켜거나 끈다
uint32_t	m_tick	틱 번호. 매 틱 1 씩 커지고, 게임이 새로 시작돼도 0 으로 되돌리지 않는다

함수

리턴 타입	함수명	인자	역할
static DWORD WINAPI	GameLoopThread	LPVOID param: CGameLoop 포인터	게임 루프 스레드가 시작되면 가장 먼저 실행되는 함수다. Run 을 부른다.
void	Run	없음	게임 루프 스레드가 하는 일 전체다. ① 시작 신호를 기다린다(WaitForSingleObject) ② PrepareGame ③ 게임이 끝날 때까지 WaitNextTick -> RunTick -> IsGameOver 를 반복 ④ SendResult ⑤ ReturnToLobby ⑥ ①로 돌아간다.
bool	PrepareGame	없음	게임 한 판을 준비한다. 성공하면 true. 하는 순서: ① 참가자 확인(GetParticipants) ② 맵(1~3)과 각자의 기체를 무작위로 정한다 ③ 맵과 World 를 만든다 ④ 남은 입력 지우기(ClearInputs) ⑤ 지금의 'E 누른 횟수'를 m_lastShieldCount 에 기록 ⑥ 전원에게 SC_GameStart 를 보낸다.

리턴 타입	함수명	인자	역할
void	WaitNextTick	없음	m_timer.Tick(30.0f)를 불러, 지난 틱부터 1/30 초가 될 때까지 기다린다. Sleep 은 쓰지 않는다.
void	RunTick	없음	틱 한 번을 진행한다. ① m_tick 을 1 올린다 ② 4 명의 입력을 복사해 온다(CopyInputs) ③ UpdateWorld ④ 이번 틱에 죽은 것이 있으면 SendEvents ⑤ SendStates.
void	UpdateWorld	const CS_Input inputs[MAX_PLAYERS]: 4 명의 입력; `std::vector<SC_Event>&` outEvents: 이번 틱의 사망 목록을 담아 돌려받을 곳	입력대로 기체를 움직이고 적 AI, 총알, 피격을 1/30 초만큼 계산한다. 이번 틱에 죽은 플레이어는 유령으로 바꾸고, 죽은 플레이어와 적을 outEvents 에 넣는다.
PlayerInput	ToPlayerInput	const CS_Input& input: 네트워크로 받은 입력; uint8_t playerId: 그 입력을 보낸 플레이어 번호	네트워크로 받은 입력을 World 가 쓰는 입력 모양(PlayerInput, 5-2)으로 바꿔 돌려준다. 'E 누른 횟수'가 지난 틱과 다르면 '이번에 E 를 눌렀다'(shieldTogglePressed = true)로 바꾼다.
void	SendEvents	const `std::vector<SC_Event>&` events: 이번 틱의 사망 목록	사망 이벤트(SC_Event)를 하나씩 전원에게 TCP 로 보낸다.
void	SendStates	없음	상태 패킷 3 종(플레이어, 적, 총알)을 채워서, UDP 주소를 아는 클라마다 보낸다(sendto).
int	FillPlayerStates	SC_PlayerStates& out: 채울 패킷	모든 참가자의 상태를 채우고, 실제로 보낼 바이트 수(8 + 20 × 인원)를 돌려준다.
int	FillEnemyStates	SC_EnemyStates& out: 채울 패킷	살아 있는 적만 채우고, 보낼 바이트 수(8 + 17 × 적 수)를 돌려준다.
int	FillBulletStates	SC_BulletStates& out: 채울 패킷	날아가는 총알만 채우고, 보낼 바이트 수(8 + 12 × 총알 수)를 돌려준다.
bool	IsGameOver	uint8_t& outWinner: 승자 번호를 담아 돌려받을 변수	살아 있는 플레이어가 1 명 이하면 true 를 돌려준다. outWinner 에는 살아남은 1 명의 PlayerId 를, 아무도 없으면(동시에 다 죽음) 255 를 넣는다.
void	SendResult	uint8_t winner: 승자 번호. 없으면 255	게임 결과(SC_GameResult)를 전원에게 보낸다.

리턴 타입	함수명	인자	역할
void	ReturnToLobby	없음	World 와 맵을 지운다 -> ResetLobby -> 전원에게 로비 상태를 보낸다(BroadcastLobbyState).

3-5. 게임 루프가 부르는 판정 클래스 (기존 클래스에 더함)

베이스 게임에 있는 계산용 클래스들이다. 서버 게임 루프 스레드에서 부르는 함수만 적었다.

클래스	리턴 타입	함수명	인자	역할
CTerrainGameMap	bool	LoadMapData (기존)	없음	맵 번호에 맞는 지형 높이 파일을 읽는다. 성공하면 true.
CTerrainGameMap	void	LoadSpawnPositions (기존)	없음	적이 나타날 위치 목록과 시작 위치를 채운다.
CTerrainGameMap	XMFLOAT3	GetPlayerStartPosition (새)	int index: 플레이어 순서 0~3	index 번 플레이어의 시작 위치를 돌려준다. 맵마다 4 곳이 있다.
CGameWorld	CPlayer*	SpawnPlayer (기존)	EPlayerType playerType: 기체 종류(헬기, 탱크); const XMFLOAT3& position: 놓을 위치; const CGameMap* gameMap: 맵(땅 높이를 알기 위해)	기체를 만들어 땅 위에 놓고, 만든 기체를 돌려준다.
CGameWorld	void	SpawnEnemies (기존)	const CGameMap* gameMap: 맵	맵에 적힌 순서대로 적을 만든다. 이 순서가 적 번호(enemyIndex)가 된다.
CGameWorld	void	BuildBulletPool (기존)	int bulletCount: 미리 만들 총알 수(64)	총알을 미리 만들어 두고 꺼서 보관한다. 쏠 때마다 꺼진 총알을 켜서 쓴다.
CGameWorld	void	UpdatePlayer (새)	CPlayer* player: 움직일 기체; float deltaTime: 진행할 시간(1/30 초); const PlayerInput& input: 이 기체의 입력; float cameraYaw: 이 플레이어 카메라의 좌우 방향(도). 앞으로 가는 방향의 기준; const CGameMap* gameMap: 맵	한 기체의 이동과 쉴드를 처리한다. 유령은 이동만 한다.

클래스	리턴 타입	함수명	인자	역할
CGameWorld	void	HandlePlayerFire (새)	CPlayer* player: 쏠 기체; uint8_t ownerPlayerId: 쏜 플레이어 번호; const PlayerInput& input: 이 기체의 입력	발사 버튼을 누르고 있고 다시 쏠 수 있는 시간이 되었으면 총알을 쏜다. 유령은 쏘지 않는다.
CGameWorld	void	UpdateEnemies (기존)	float deltaTime: 진행할 시간; const CGameMap* gameMap: 맵	적이 가장 가까운 살아 있는 플레이어를 쫓아가 쏜다.
CGameWorld	void	UpdateBullets (기존)	float deltaTime: 진행할 시간; const CGameMap* gameMap: 맵	총알을 움직이고, 최대 거리를 넘거나 땅에 닿으면 끈다.
CGameWorld	void	CheckBulletHits (새)	`std::vector<CEnemy*>&` outDeadEnemies: 이번에 죽은 적을 담아 돌려받을 곳; `std::vector<CPlayer*>&` outDeadPlayers: 이번에 죽은 플레이어를 담아 돌려받을 곳	총알이 맞았는지 확인하고 피해를 준다. 이번에 죽은 적과 플레이어를 돌려준다. 플레이어가 쏜 총알은 다른 플레이어도 맞히고(자기 총알은 안 맞음), 쉴드가 켜져 있으면 피해가 없다.
CGameWorld	int	CountAlivePlayers (새)	CPlayer*& outLastAlive: 마지막 생존자를 담아 돌려받을 변수	살아 있는 플레이어 수를 돌려주고, 살아 있는 플레이어 하나를 outLastAlive 에 넣는다(1 명 남았을 때 그 사람이 승자). const 함수: 멤버 값을 바꾸지 않는다.
CGameWorld	void	Clear (기존)	없음	기체, 적, 총알을 모두 지운다.
CPlayer	void	SetGhost (새)	bool ghost: true 면 유령으로	유령(죽은 뒤 관전만 하는 상태)으로 바꾼다. 유령은 움직일 수만 있고, 쏘거나 맞지 않는다. 클라에서는 내 기체가 아닌 유령은 그리지 않는다.
CPlayer	bool	IsGhost (새)	없음	유령이면 true 를 돌려준다(const 함수: 멤버 값을 바꾸지 않음).

클래스	리턴 타입	함수명	인자	역할
CBullet	void	Fire (기존, 바뀜)	const XMFLOAT3& startPosition: 나가는 위치; const XMFLOAT3& direction: 나가는 방향; EBulletOwner newOwner: 플레이어 총알인지 적 총알인지; uint8_t ownerPlayerId: 쏜 플레이어 번호(새로 더하는 인자)	꺼져 있던 총알을 켜서 쏜다. 쏜 플레이어 번호를 기록해 두어 자기 총알에는 맞지 않게 한다.
CBullet	uint8_t	GetOwnerPlayerId (새)	없음	이 총알을 쏜 플레이어 번호를 돌려준다(const 함수: 멤버 값을 바꾸지 않음).

더하는 멤버 변수: CPlayer 의 bool ghost(유령 상태인가(새로 더하는 값)), CBullet 의 uint8_t ownerPlayerId(쏜 플레이어 번호(새로 더하는 값). 적이 쏜 총알이면 255).

4. 클라 클래스

4-1. CNetworkClient (새)

클라의 네트워크 일을 모두 맡는다. 소켓 2 개(TCP, UDP), 수신 스레드 2 개, 이벤트 큐, 최신 상태를 가지고 있다. CGameFramework 가 하나 가지고 있다.

멤버 변수

형식	이름	뜻
SOCKET	m_tcpSock	서버와 연결된 TCP 소켓
SOCKET	m_udpSock	UDP 소켓. 포트 0 으로 bind 한다(0 = 운영체제가 빈 포트를 골라 줌)
sockaddr_in	m_serverUdpAddr	입력을 보낼 서버의 UDP 주소(SERVER_IP 의 9001 번 포트)
HANDLE	m_tcpThread, m_udpThread	TCP 수신, UDP 수신 스레드의 핸들
`std::vector<NetEvent>`	m_eventQueue	이벤트 큐. TCP 로 받은 패킷을 도착 순서대로 쌓아 둔다
CRITICAL_SECTION	m_queueLock	큐 잠금. TCP 수신 스레드(넣기)와 메인 스레드(꺼내기)가 동시에 큐를 만지지 않게 한다
LatestStates	m_latest	UDP 로 받은 최신 상태 3 종(5-2)
CRITICAL_SECTION	m_stateLock	상태 잠금. UDP 수신 스레드(덮어쓰기)와 메인 스레드(복사)가 동시에 만지지 않게 한다
uint32_t	m_lastTick[3]	상태 종류(플레이어, 적, 총알)별로 마지막으로 받은 tick 번호. UDP 수신 스레드만 쓴다

형식	이름	뜻
uint8_t	m_myPlayerId	내 PlayerId. 접속이 승인될 때 받는다
uint32_t	m_inputSequence	마지막으로 보낸 입력의 일련번호. 보낼 때마다 1 씩 올리고 되돌리지 않는다

함수

리턴 타입	함수명	인자	역할	스레드
bool	Connect	const char* serverIp: 서버 IP 문자열(SERVER_IP)	서버에 TCP 로 연결한다. 성공하면 true, 실패하면 만든 것을 정리하고 false 를 돌려준다. 하는 순서: ① 윈속 시작(WSAStartup) ② TCP 소켓 만들기 ③ 서버의 9000 번 포트에 연결(connect).	클라 메인
bool	StartReceive	없음	패킷을 받을 준비를 한다. 성공하면 true. 하는 순서: ① UDP 소켓을 만들고 포트 0 으로 bind 한다(bind 하지 않으면 recvfrom 이 WSAEINVAL 오류를 낸다) ② 서버의 UDP 주소를 저장한다 ③ TCP 수신, UDP 수신 스레드를 시작한다.	클라 메인
void	SendReady	없음	준비 패킷(CS_Ready)을 TCP 로 보낸다.	클라 메인
void	SendInput	int8_t moveForward: 앞뒤 이동: -1(S), 0, 1(W); int8_t moveRight: 좌우 이동: -1(A), 0, 1(D); float cameraYaw: 카메라의 좌우 방향(도); uint8_t fire: 좌클릭을 누르고 있으면 1; uint8_t shieldPressCount: E 를 누른 누적 횟수	내 PlayerId 와 새 일련번호(지난 번호 + 1)를 붙여 입력 패킷(CS_Input)을 만들고 서버로 UDP 전송한다(sendto).	클라 메인
void	PopEvents	`std::vector<NetEvent>&` out: 꺼낸 이벤트를 받을 빈 목록	큐 잠금을 잡고 큐와 out 을 통째로 맞바꾼다(swap). 그러면 out 에는 쌓여 있던 이벤트가 전부 들어오고 큐는 빈다. 잠금은 맞바꾸는 짧은 순간만 잡는다.	클라 메인
void	CopyLatestStates	LatestStates& out: 복사해 받을 곳	상태 잠금을 잡고 최신 상태 3 종을 out 에 복사한 뒤, '새 값 있음' 표시(hasNew…)를 끈다.	클라 메인
void	SetMyPlayerId	uint8_t id: 승인 때 받은 내 번호	내 PlayerId 를 저장한다.	클라 메인

리턴 타입	함수명	인자	역할	스레드
uint8_t	GetMyPlayerId	없음	내 PlayerId 를 돌려준다(const 함수: 멤버 값을 바꾸지 않음).	클라 메인
void	Disconnect	없음	소켓을 닫는다(그러면 수신 스레드의 받기가 실패해서 스레드가 끝난다) -> 스레드가 끝나길 기다린다 -> 윈속을 정리한다(WSACleanup).	클라 메인
static DWORD WINAPI	TcpRecvThread	LPVOID param: CNetworkClient 포인터	TCP 수신 스레드가 시작되면 가장 먼저 실행되는 함수다. TcpRecvLoop 를 부른다.	클라 TCP 수신
static DWORD WINAPI	UdpRecvThread	LPVOID param: CNetworkClient 포인터	UDP 수신 스레드가 시작되면 가장 먼저 실행되는 함수다. UdpRecvLoop 를 부른다.	클라 UDP 수신
void	TcpRecvLoop	없음	서버가 보내는 TCP 패킷을 하나씩 받아(RecvPacket) NetEvent 로 만들어 큐에 넣기를 반복한다. 연결이 끊기면 끝난다.	클라 TCP 수신
void	UdpRecvLoop	없음	UDP 상태 패킷 받기를 반복한다. 크기 검사와 tick 검사(그 종류의 마지막 tick 보다 작거나 같으면 버림)를 통과하면 그 종류의 최신 상태에 덮어쓰고 '새 값 있음' 표시를 켠다.	클라 UDP 수신

4-2. CGameFramework (기존)

클라 메인 스레드에서 매 프레임을 진행하고 화면(씬)을 바꾼다. 받은 TCP 패킷을 꺼내 지금 화면에 나눠 준다. 4-2~4-8 의 함수는 모두 클라 메인 스레드에서 실행된다.

멤버 변수 (새로 더하는 것)

형식	이름	뜻
CNetworkClient	m_network	네트워크를 맡는 객체(4-1)
SC_LobbyState	m_lastLobbyState	가장 최근에 받은 로비 상태. 어느 화면에서 받든 저장해 두었다가 로비 화면을 만들 때 넘긴다
SC_GameStart	m_gameStart	받은 게임 시작 정보(맵, 참가자, 시작 위치). 게임 화면을 만들 때 쓴다

함수

리턴 타입	함수명	인자	역할
void	FrameAdvance (기존, 바뀜)	없음	한 프레임을 진행한다. ① gameTimer.Tick(30.0f)로 지난 프레임부터 1/30 초가 될 때까지 기다림 ② ProcessInput(키 입력) ③ ProcessNetEvents(받은 TCP 패킷 처리) ④ AnimateObjects(움직임 갱신) ⑤ 그리기 ⑥ Present(그린 것을 화면에 보이기) ⑦ CheckSceneChange(화면 바꾸기 요청 처리).
void	ProcessNetEvents (새)	없음	PopEvents 로 큐를 꺼내 하나씩 종류를 보고 처리한다. SC_LobbyState 는 m_lastLobbyState 에 저장하고, 지금이 로비 화면이면 SetLobbyState 로 바로 반영한다. SC_GameStart 는 m_gameStart 에 저장하고 게임 화면으로 바꾼다. 그 밖의 패킷은 지금 화면의 OnNetEvent 에 넘긴다.
void	ChangeScene (기존, 바뀜)	ESceneType nextScene: 바꿀 화면 종류; bool buildDirectResources: 그리기 자원을 새로 만들지	화면을 바꾼다. 게임 화면이면 m_gameStart 로 GameSessionConfig(5-2)를 채워 넘기고, 로비 화면이면 m_lastLobbyState 를 넘긴다. 새 화면에 SetNetworkClient 로 네트워크 객체를 넣어 준다.
void	CheckSceneChange (기존)	없음	화면이 남겨 둔 '바꿔 달라'는 요청(requestedScene)이 있으면 ChangeScene 을 부른다. 그리는 도중에 화면이 바뀌지 않도록 Present 뒤에서 부른다.

4-3. CSceneBase (기존) — 화면 4 개의 부모 클래스

시작, 로비, 게임, 결과 화면이 모두 이 클래스를 물려받는다. 여기에 더한 것은 화면 4 개가 모두 가진다.

멤버 변수 (새로 더하는 것)

형식	이름	뜻
CNetworkClient*	m_network	네트워크 객체 포인터. 화면에서 패킷을 보낼 때 쓴다

함수

리턴 타입	함수명	인자	역할
virtual void	OnNetEvent (새)	const NetEvent& event: 받은 TCP 패킷 하나	화면이 TCP 패킷을 받는 함수다. 기본은 아무것도 하지 않고, 패킷이 필요한 화면(시작, 게임 화면)만 다시 만든다(override).
void	SetNetworkClient (새)	CNetworkClient* network: 네트워크 객체 포인터	m_network 에 넣는다.

4-4. CStartScene (기존) — 시작 화면

게임을 켜면 처음 나오는 화면이다. 제목을 클릭하면 서버에 접속한다.

함수

리턴 타입	함수명	인자	역할
bool	OnProcessingMouseMessage (기존, 바뀜)	HWND hWnd: 창 핸들; UINT message: 마우스 메시지 종류(클릭 등); WPARAM wParam: 메시지 추가 정보; LPARAM lParam: 마우스 좌표	제목 글자를 클릭하면 Connect(SERVER_IP)를 부르고, 성공하면 StartReceive 를 부른다. 이 클릭을 처리했으면 true.
void	OnNetEvent (새)	const NetEvent& event: 받은 TCP 패킷 하나	접속 결과(SC_ConnectResult)를 처리한다. accepted 가 1(승인)이면 SetMyPlayerId(받은 번호) 후 로비 화면을 요청한다. 0(거절)이면 제목 글자를 빨간색으로 바꾸고 Disconnect 를 부른다. 시작 화면에 그대로 남아 다시 클릭할 수 있다.

4-5. CMapSelectScene (기존) — 로비 화면

게임 전에 모여서 준비 버튼을 누르는 화면이다. 원래 맵 선택 화면이던 것을 로비로 쓴다.

멤버 변수 (새로 더하는 것)

형식	이름	뜻
SC_LobbyState	m_lobbyState	화면에 그릴 로비 상태(누가 접속했고 누가 준비했나)
bool	m_readySent	준비 버튼을 이미 눌러 CS_Ready 를 보냈으면 true. 두 번 보내지 않으려고 둔다

함수

리턴 타입	함수명	인자	역할
void	BuildObjects (기존, 바뀜)	int clientWidth: 창 너비; int clientHeight: 창 높이	로비 화면을 만든다. m_readySent = false 로 시작해서 준비 버튼이 눌리는 상태가 된다.
bool	OnProcessingMouseMessage (기존, 바뀜)	HWND hWnd: 창 핸들; UINT message: 마우스 메시지 종류(클릭 등); WPARAM wParam: 메시지 추가 정보; LPARAM lParam: 마우스 좌표	준비 버튼(Start 글자)을 처음 누르면 SendReady 를 부르고 버튼을 어둡게 한다(m_readySent = true). 이 클릭을 처리했으면 true.
void	SetLobbyState (새)	const SC_LobbyState& state: 받은 로비 상태	받은 로비 상태를 저장하고 자리 4 칸의 색을 바꾼다. 빈 자리는 회색, 접속은 흰색, 준비는 초록색이고 내 자리는 크게 그린다.
void	RenderDirect (기존, 바뀜)	ID3D12GraphicsCommandList* commandList: DirectX12 그리기 명령을 담는 객체	준비 버튼과 자리 4 칸을 그린다.

4-6. CGameScene (기존) — 게임 화면

게임 중 화면이다. 계산(이동, 피격, 승패)은 서버가 하므로, 이 화면은 입력을 보내고 서버가 보낸 값을 그대로 보여 주기만 한다.

멤버 변수 (새로 더하는 것)

형식	이름	뜻
CPlayer*	m_playerById[MAX_PLAYERS]	PlayerId 로 그 플레이어의 기체를 찾는 표
uint8_t	m_shieldPressCount	E 를 누른 누적 횟수. 누를 때마다 1 씩 올려서 서버로 보낸다

함수

리턴 타입	함수명	인자	역할
(생성자)	CGameScene (기존, 바뀜)	const GameSessionConfig& config: 서버가 정한 맵, 참가자, 내 번호	서버가 정한 맵과 참가자 정보로 게임 화면 객체를 만든다.
void	BuildObjects (기존, 바뀜)	int clientWidth: 창 너비; int clientHeight: 창 높이	맵, 참가자 전원의 기체, 적, 총알을 만들고 그중 내 기체를 정한다.
bool	ProcessInput (기존, 바뀜)	UCHAR* keyBuffer: 키보드 256 개 키의 눌림 상태	키보드, 마우스로 이번 프레임의 입력을 만들고 SendMyInput 을 부른다.
void	SendMyInput (새)	없음	이번 프레임 입력을 패킷 값으로 바꿔(이동 -1/0/1, 카메라 좌우 방향, 발사 0/1, E 누른 누적 횟수) SendInput 을 부른다.
void	OnNetEvent (새)	const NetEvent& event: 받은 TCP 패킷 하나	사망 이벤트(SC_Event): 죽은 것이 플레이어면 그 기체를 유령으로 바꾸고, 적이면 지운다. 둘 다 그 위치에 폭발을 보여 준다. 게임 결과(SC_GameResult): 승자 번호가 내 번호면 승리 화면, 아니면(255 포함) 패배 화면을 요청한다.
void	AnimateObjects (기존, 바뀜)	float deltaTime: 지난 프레임부터 흐른 시간(초)	CopyLatestStates 로 최신 상태를 가져와 ApplyStates 를 부른 뒤, 카메라, 쉴드 색, 폭발 연출을 갱신한다.
void	ApplyStates (새)	const LatestStates& s: 가져온 최신 상태	새 값이 온 종류만 반영한다. 플레이어는 위치, 방향, 체력, 쉴드, 유령 여부, 적은 위치, 방향, 총알은 위치를 넣는다. 상태 패킷에 없는 적은 마지막 위치 그대로 둔다(적을 지우는 것은 사망 이벤트뿐).
void	RenderDirect (기존, 바뀜)	ID3D12GraphicsCommandList* commandList: DirectX12 그리기 명령을 담는 객체	지형, 기체(유령은 내 것만), 적, 총알, 폭발을 그린다.

4-7. CEndScene (기존) — 결과 화면

게임이 끝나면 승리 또는 패배를 보여 주는 화면이다.

함수

리턴 타입	함수명	인자	역할
(생성자)	CEndScene (기존)	bool victory: true 면 승리, false 면 패배	true 면 'YOU WIN!', false 면 'LOSE'를 보여 주는 화면을 만든다.
bool	OnProcessingMouseMessage (기존, 바뀜)	HWND hWnd: 창 핸들; UINT message: 마우스 메시지 종류(클릭 등); WPARAM wParam: 메시지 추가 정보; LPARAM lParam: 마우스 좌표	글자를 클릭하면 로비 화면(MapSelect)으로 가는 전환을 요청한다.

4-8. 그 밖의 클라 함수

클라 메인 스레드에서 쓰는, 베이스 게임 클래스에 더하거나 그대로 쓰는 함수다.

클래스	리턴 타입	함수명	인자	역할
CGameTimer	void	Tick (기존)	float fLockFPS = 0.0f: 1 초에 몇 번으로 맞출지. 30 을 넣는다	지난 호출부터 1/fLockFPS 초(30 이면 1/30 초)가 될 때까지 기다린다. 클라 FrameAdvance 와 서버 WaitNextTick 이 같이 쓴다.
CPlayer	void	SetHealth (새)	int health: 서버에서 받은 체력	받은 체력을 넣는다(화면 표시용).
CPlayer	void	SetShieldEnabled (새)	bool enabled: 서버에서 받은 쉴드 상태	받은 쉴드 상태를 넣는다(화면 표시용).
CPlayer	void	SetGhost (새)	bool ghost: true 면 유령으로	유령으로 바꾼다(3-5 와 같은 함수). 내 기체가 아니면 그리지 않는다.

5. 공통 함수와 구조체

5-1. 공통 함수 (NetUtil.h, 클라, 서버가 같이 씀)

리턴 타입	함수명	인자	역할
bool	SendAll	SOCKET sock: 보낼 TCP 소켓; const char* data: 보낼 데이터; int len: 보낼 바이트 수	len 바이트가 다 나갈 때까지 send()를 반복한다(send 한 번에 다 안 나갈 수 있어서). 오류가 나면 false. 서버에서는 SendPacket 안에서만 부른다. 단, 거절 결과는 자리가 없어서 SendAll 로 바로 보낸다.
bool	RecvPacket	SOCKET sock: 받을 TCP 소켓; char* buf: 받은 패킷을 담을 공간; int bufSize: buf 의 크기(바이트)	TCP 로 패킷 하나를 받는다. 연결이 끊겼거나(recv 가 0) 오류이거나 size 가 이상하면 false. 받을 때는 recv(..., MSG_WAITALL)를 써서 요청한 바이트가 다 올 때까지 기다린다. 하는 순서: ① 헤더 3 바이트를 다 받는다 ② 헤더의 size 가 3 이상 bufSize 이하인지 확인한다 ③ 나머지 size - 3 바이트를 다 받는다.

5-2. 구조체와 상수

이름	멤버 (형식 이름: 뜻)	쓰는 곳
struct NetEvent (새, NetUtil.h)	TCP 로 받은 패킷 하나. TCP 수신 스레드가 만들어 큐에 넣고, 메인 스레드가 꺼낸다. EPacketType type: 패킷 종류(헤더의 type); `std::vector<char>` data: 받은 패킷 전체(헤더 포함)	클라: TCP 수신 스레드 -> 큐 -> 메인 스레드
struct LatestStates (새, NetUtil.h)	UDP 로 받은 최신 상태 3 종. UDP 수신 스레드가 덮어쓰고, 메인 스레드가 복사해 간다. SC_PlayerStates players: 최신 플레이어 상태; SC_EnemyStates enemies: 최신 적 상태; SC_BulletStates bullets: 최신 총알 상태; bool hasNewPlayers, hasNewEnemies, hasNewBullets: 종류별 '새 값 있음' 표시. UDP 수신 스레드가 덮어쓸 때 켜고, CopyLatestStates 가 끈다	클라: UDP 수신 스레드 -> 메인 스레드
struct PlayerInput (기존, PlayerInput.h)	한 프레임의 입력. 클라는 키 입력을 모을 때, 서버는 World 를 계산할 때 쓴다. float moveForward: 앞뒤 이동 -1 ~ 1 (S/W); float moveRight: 좌우 이동 -1 ~ 1 (A/D); float lookYawDelta, lookPitchDelta: 마우스 이동량. 클라 카메라에만 쓴다; bool fire: 좌클릭을 누르고 있으면 true; bool shieldTogglePressed: 이번에 E 를 눌렀으면 true	클라 입력 만들기, 서버 World 계산
struct GameSessionConfig (기존 + 더함, GameSessionConfig.h)	클라가 게임 화면을 만들 때 넘기는 설정이다. int mapNumber: 맵 번호 1~3; EPlayerType playerType: 내 기체 종류; EGameMode gameMode: 게임 규칙. FreeForAll(개인전)로 쓴다; uint8_t myPlayerId: 내 PlayerId (새); uint8_t playerCount: 이번 게임 참가 인원 (새); PlayerStartInfo players[MAX_PLAYERS]: 참가자별 번호, 기체 종류, 시작 위치 (새)	클라 게임 화면 만들기
enum class EPlayerType { None, Helicopter, Tank } (기존, PlayerType.h)	기체 종류. 패킷의 playerType 값(0, 1, 2)과 순서가 같다. 게임 코드에 이미 있으므로 Protocol.h 에서는 빼고 이 파일을 include 한다(제안).	클라, 서버
constexpr const char* SERVER_IP = "127.0.0.1"; (새, 클라 전역 상수)	접속할 서버의 IP. 127.0.0.1 은 내 PC 를 뜻한다. 시연 때는 서버 PC 의 IP 로 바꿔서 빌드한다.	시작 화면(Connect)

6. 순서도 따라가기

전체 순서도의 상자를 위에서 아래 순서로, 그 일을 하는 함수에 연결했다. 함수의 자세한 설명은 3~5 장에 있다.

판단 상자(◇)는 예/아니오로 갈라지는 상자다. 이런 줄은 '함수' 칸에 그 판단을 하는 함수를, '하는 일' 칸에 무엇을 보고 예/아니오를 정하는지를 적었다. 예: '빈 자리 있고 로비 중인가?'는 TryRegisterSeat()가 돌려준 값을 보고, 255 가 아니면 예, 255 면 아니오다.

6-1. 접속

순서도 상자	스레드	함수	하는 일
서버 준비	서버 메인	CGameServer::Start()	소켓, 잠금, 시작 이벤트를 준비하고 UDP 수신, 게임 루프 스레드를 시작한다.
게임 루프는 시작 신호 대기	서버 게임 루프	CGameLoop::Run()	WaitForSingleObject(m_startEvent)에서 시작 신호가 올 때까지 기다린다.
시작 화면	클라 메인	CStartScene::OnProcessingMouseMessage()	제목을 클릭하면 Connect 를 부른다. 윈속 시작(WSAStartup)은 Connect 안에서 맨 먼저 한다.
서버와 TCP 연결	클라 메인	CNetworkClient::Connect()	TCP 소켓을 만들고 서버(SERVER_IP 의 9000 번 포트)에 연결한다.
◇ 연결됐나?	클라 메인	CNetworkClient::Connect()	돌려준 값이 true 면 예 -> 접속 준비. false 면 아니오 -> 시작 화면으로.
연결 받아들이기	서버 메인	CGameServer::AcceptLoop()	접속을 받고(accept) -> 그 클라를 맡을 TCP 통신 스레드를 만들고 -> 다시 접속을 기다린다.
접속 준비	클라 메인	CNetworkClient::StartReceive()	UDP 소켓을 포트 0 으로 bind 하고 수신 스레드 2 개(TCP, UDP)를 시작한다.
TCP 통신 스레드 시작	서버 TCP 통신	CGameServer::TcpCommThread() -> TcpCommLoop()	시작하자마자 자리 등록을 시도한다.
◇ 빈 자리 있고 로비 중인가?	서버 TCP 통신	CGameServer::TryRegisterSeat()	돌려준 값이 255 가 아니면 예 -> 자리 등록. 255 면 아니오 -> 거절.
자리 없음 또는 게임 중 -> 거절	서버 TCP 통신	CGameServer::TryRegisterSeat()	255 를 돌려주고, 거절 이유(ServerFull 또는 AlreadyStarted)를 outReason 에 적는다.
자리 등록	서버 TCP 통신	CGameServer::TryRegisterSeat()	가장 앞의 빈 자리에 이 클라를 적고, 그 자리 번호를 PlayerId 로 돌려준다.
결과, 로비 상태 전송	서버 TCP 통신	CGameServer::SendConnectResult(), BroadcastLobbyState()	이 클라에게 접속 결과를 보내고, 승인이면 전원에게 로비 상태를 보낸다.
◇ 승인했나?	서버 TCP 통신	CGameServer::TcpCommLoop()	TryRegisterSeat()가 돌려준 값이 255 가 아니면 예 -> 다음 패킷 기다리기. 255 면 아니오 -> 소켓 닫고 스레드 종료.
소켓 닫고 스레드 종료	서버 TCP 통신	CGameServer::SendConnectResult()	거절 결과를 보낸 뒤 소켓을 닫는다(closesocket). TcpCommLoop()가 끝나면 스레드도 끝난다.

순서도 상자	스레드	함수	하는 일
TCP 받기 (결과, 로비 상태)	클라 TCP 수신	CNetworkClient::TcpRecvLoop()	받은 패킷을 큐에 넣는다.
응답 기다리기	클라 메인	CGameFramework::ProcessNetEvents()	매 프레임 큐를 꺼내 본다. 비어 있으면 다음 프레임에 다시 본다.
◇ 접속 결과가 왔나?	클라 메인	CGameFramework::ProcessNetEvents()	꺼낸 것 중에 SC_ConnectResult 가 있으면 예 -> CStartScene::OnNetEvent()로 넘긴다. 없으면 아니오 -> 응답 기다리기.
◇ 승인인가?	클라 메인	CStartScene::OnNetEvent()	패킷의 accepted 가 1 이면 예 -> SetMyPlayerId 후 로비 화면. 0 이면 아니오 -> 거절 표시.
거절 표시 후 시작 화면으로	클라 메인	CStartScene::OnNetEvent()	제목 글자를 빨간색으로 바꾸고 Disconnect 를 부른다.

6-2. 로비와 게임 시작

순서도 상자	스레드	함수	하는 일
로비 화면	클라 메인	CMapSelectScene::RenderDirect()	준비 버튼과 자리 4 칸(누가 접속했고 누가 준비했나)을 그린다.
◇ 대기 버튼을 처음 눌렀나?	클라 메인	CMapSelectScene::OnProcessingMouseMessage()	준비 버튼을 눌렀고 m_readySent 가 false(아직 안 보냄)면 예 -> 준비 알리기. 아니면 아니오 -> 큐 확인.
준비 알리기 (한 번만)	클라 메인	CNetworkClient::SendReady()	CS_Ready 를 보내고 m_readySent 를 true 로 바꿔 버튼을 끈다.
다음 패킷 기다리기	서버 TCP 통신	CGameServer::TcpCommLoop()	RecvPacket()에서 이 클라의 패킷이 올 때까지 기다린다.
준비 기록	서버 TCP 통신	CGameServer::SetReady()	그 자리를 '준비 완료'로 바꾼다.
로비 상태 알리기	서버 TCP 통신	CGameServer::BroadcastLobbyState()	전원에게 로비 상태를 보낸다.
TCP 받기 (로비 상태) -> 큐 확인	클라 TCP 수신 -> 클라 메인	CNetworkClient::TcpRecvLoop() -> CGameFramework::ProcessNetEvents() -> CMapSelectScene::SetLobbyState()	받은 로비 상태로 자리 4 칸을 다시 그린다.
◇ 모두 준비 + 2 명 이상?	서버 TCP 통신	CGameServer::TryStartGame()	돌려준 값이 true 면 예 -> 게임 시작 신호. false 면 아니오 -> 다음 패킷 기다리기.
게임 시작 신호	서버 TCP 통신	CGameServer::TcpCommLoop()	SetEvent(m_startEvent)로 게임 루프 스레드를 깨우고, 다시 다음 패킷을 기다린다.
게임 준비	서버 게임 루프	CGameLoop::PrepareGame()	맵과 기체를 정하고 World 를 만든 뒤 전원에게 SC_GameStart 를 보낸다.

순서도 상자	스레드	함수	하는 일
TCP 받기 (게임 시작)	클라 TCP 수신	CNetworkClient::TcpRecvLoop()	받은 패킷을 큐에 넣는다.
◇ 게임 시작 패킷 왔나?	클라 메인	CGameFramework::ProcessNetEvents()	꺼낸 것 중에 SC_GameStart 가 있으면 예 -> m_gameStart 에 저장하고 게임 화면으로. 없으면 아니오 -> 로비 화면.
게임 화면 만들기	클라 메인	CGameFramework::ChangeScene() -> CGameScene 생성자, CGameScene::BuildObjects()	받은 맵, 기체, 시작 위치로 게임 화면을 만든다. 계산(판정)은 하지 않는다.

6-3. 플레이 (클라, 서버 둘 다 1/30 초마다)

순서도 상자	스레드	함수	하는 일
입력 만들기	클라 메인	CGameScene::ProcessInput()	키보드, 마우스로 이동, 발사, 쉴드, 카메라 방향을 모은다.
입력 보내기	클라 메인	CGameScene::SendMyInput() -> CNetworkClient::SendInput()	입력 패킷(CS_Input)을 서버로 UDP 전송한다.
입력 받기 (늦게 온 건 버림)	서버 UDP 수신	CGameServer::UdpRecvLoop()	받은 뒤 검사한다. 일련번호(sequence)가 그 칸의 lastSequence 보다 작거나 같으면 늦게 온 것이라 버린다.
입력, 주소 저장	서버 UDP 수신	CGameServer::UdpRecvLoop()	그 플레이어 칸에 입력, 일련번호, 주소를 덮어쓰고 다시 받는다.
마지막 입력 + UDP 주소, 4 명분	(데이터)	CGameServer::m_inputs[4]	입력 잠금으로 지킨다.
틱 시작	서버 게임 루프	CGameLoop::WaitNextTick(), RunTick() -> CGameServer::CopyInputs()	1/30 초가 될 때까지 기다린 뒤 4 명분 입력을 복사해 온다.
게임 세계 갱신	서버 게임 루프	CGameLoop::UpdateWorld()	이동, 적 AI, 총알, 피격을 계산하고, 죽은 플레이어는 유령으로 바꾼다.
◇ 이번 틱에 죽은 사람?	서버 게임 루프	CGameLoop::RunTick()	UpdateWorld()가 채운 사망 목록(outEvents)이 비어 있지 않으면 예 -> 사망 알리기. 비어 있으면 아니오 -> 상태 보내기.
사망 알리기	서버 게임 루프	CGameLoop::SendEvents()	사망 이벤트(SC_Event)를 전원에게 TCP 로 보낸다.
상태 보내기	서버 게임 루프	CGameLoop::SendStates()	상태 3 종을 클라마다 UDP 로 보낸다(입력을 받을 때 저장해 둔 주소로).
◇ 살아남은 사람 1 명 이하?	서버 게임 루프	CGameLoop::IsGameOver()	돌려준 값이 true 면 예 -> 결과 알리기. false 면 아니오 -> 다음 틱.

순서도 상자	스레드	함수	하는 일
TCP 받기 (사망 이벤트)	클라 TCP 수신	CNetworkClient::TcpRecvLoop()	받은 패킷을 큐에 넣는다.
UDP 받기 -> 상태 저장	클라 UDP 수신	CNetworkClient::UdpRecvLoop()	tick 이 그 종류의 마지막 tick 보다 작거나 같으면 버리고, 아니면 최신 상태에 덮어쓴다.
받은 것 반영	클라 메인	CGameFramework::ProcessNetEvents() -> CGameScene::OnNetEvent(), CGameScene::AnimateObjects() -> ApplyStates()	큐에서 꺼낸 사망 이벤트로 유령, 적 지우기, 폭발을 처리하고, 최신 상태로 위치, 체력 등을 반영한다.
화면 그리기	클라 메인	CGameScene::RenderDirect(), CGameTimer::Tick(30.0f)	화면을 그린다. 다음 프레임의 맨 처음(Tick)에서 1/30 초가 될 때까지 기다린다.
◇ 결과 패킷 왔나?	클라 메인	CGameFramework::CheckSceneChange()	CGameScene::OnNetEvent()가 SC_GameResult 를 받아 결과 화면으로 가는 요청(requestedScene)을 남겼으면 예 -> 결과 화면. 없으면 아니오 -> 입력 만들기.

6-4. 게임 종료와 로비 복귀

순서도 상자	스레드	함수	하는 일
결과 알리기	서버 게임 루프	CGameLoop::SendResult()	마지막 생존자의 번호(없으면 255)를 SC_GameResult 로 전원에게 보낸다.
로비로 되돌리기	서버 게임 루프	CGameLoop::ReturnToLobby()	World 를 지우고 -> 준비를 모두 풀고(ResetLobby) -> 전원에게 로비 상태를 보낸다.
게임 준비로 돌아가 다음 시작 신호 대기	서버 게임 루프	CGameLoop::Run()	반복의 처음으로 돌아가 시작 신호를 기다린다.
TCP 받기 (게임 결과, 로비 상태)	클라 TCP 수신	CNetworkClient::TcpRecvLoop()	받은 패킷을 큐에 넣는다.
결과 화면	클라 메인	CGameFramework::ChangeScene() -> CEndScene 생성자	승자 번호가 내 PlayerId 면 승리, 아니면(255 포함) 패배를 보여 준다. 함께 온 로비 상태는 m_lastLobbyState 에 저장해 둔다.
화면 클릭	클라 메인	CEndScene::OnProcessingMouseMessage()	로비 화면으로 가는 전환을 요청한다.
로비 준비	클라 메인	CMapSelectScene::BuildObjects()	m_readySent 를 false 로 되돌려 준비 버튼을 다시 켠다. 서버와의 연결은 그대로다.
로비 화면으로	클라 메인	CGameFramework::ChangeScene() -> CMapSelectScene::SetLobbyState()	저장해 둔 로비 상태로 바로 그린다.

7. 스레드와 임계 영역

순서도의 상자 색(어느 스레드가 하는 일인지)과 '(… 임계 영역)' 표시를 표로 옮겼다. 서버의 World 는 서버 게임 루프 스레드만 만지고, 클라의 화면과 게임 오브젝트는 클라 메인 스레드만 만진다. 그래서 이 둘에는 잠금이 필요 없다.

7-1. 스레드 (순서도 상자 색)

순서도 색	프로그램	스레드	개수	시작되는 상자	평소에 기다리는 곳
파랑	클라	클라 메인	1	(프로그램 시작)	매 프레임 맨 처음의 CGameTimer::Tick(30.0f) — 1/30 초가 될 때까지
주황	클라	클라 TCP 수신	1	접속 준비	recv — 서버의 TCP 패킷이 올 때까지
초록	클라	클라 UDP 수신	1	접속 준비	recvfrom — 서버의 UDP 패킷이 올 때까지
파랑	서버	서버 메인	1	(프로그램 시작)	accept — 클라가 접속할 때까지
주황	서버	서버 TCP 통신	클라 1 명당 1 개, 최대 4 개	연결 받아들이기	recv('다음 패킷 기다리기') — 그 클라의 패킷이 올 때까지
초록	서버	서버 UDP 수신	1	서버 준비	recvfrom('입력 받기') — 입력 패킷이 올 때까지
보라	서버	서버 게임 루프	1	서버 준비	시작 신호가 올 때까지(WaitForSingleObject), 게임 중에는 '틱 시작'에서 1/30 초가 될 때까지(CGameTimer::Tick(30.0f))

서버 UDP 소켓은 서버 UDP 수신 스레드(받기)와 서버 게임 루프 스레드(보내기)가 같이 쓴다. UDP 를 보내는 스레드는 게임 루프 하나뿐이라 UDP 송신 잠금은 두지 않는다. 클라의 TCP 도 보내는 스레드가 클라 메인 하나뿐이라 송신 잠금이 없다.

7-2. 임계 영역 (순서도의 표시)

프로그램	잠금(임계 영역)	지키는 데이터	순서도에 표시된 상자	표시는 없지만 같은 데이터를 만지는 상자
서버	자리 잠금 (1 개, m_seatLock)	자리 4 칸, PlayerId, 준비 여부, '게임 중' 여부	자리 등록, 준비 기록	빈 자리 있고 로비 중인가?, 모두 준비 + 2 명 이상?, 로비 상태 알리기(로비 상태를 만들 때), 게임 준비, 로비로 되돌리기
서버	입력 잠금 (1 개, m_inputLock)	플레이어별 마지막 입력과 UDP 주소, 4 명분	마지막 입력 + UDP 주소, 4 명분	입력 받기, 입력, 주소 저장, 틱 시작, 상태 보내기, 자리 등록, 게임 준비
서버	송신 잠금 (자리마다 1 개, sendLock)	그 클라의 TCP 소켓	결과, 로비 상태 전송, 로비 상태 알리기, 게임 준비	사망 알리기, 결과 알리기, 로비로 되돌리기(모두 SendPacket 을 거치므로 저절로 잠긴다)
클라	큐 잠금 (m_queueLock)	이벤트 큐	TCP 받기(5 곳)	응답 기다리기, 큐 확인, 게임 시작 패킷 왔나?, 받은 것 반영, 결과 화면(메인이 큐를 통째로 맞바꿔 꺼낼 때)
클라	상태 잠금 (m_stateLock)	최신 상태 3 종	상태 저장	받은 것 반영(메인이 복사할 때)

7-3. 순서와 충돌 규칙

1. 잠금은 '동시에 만지지 않게'만 해 주고 '누가 먼저'는 정하지 않는다. '로비 상태를 보낸 다음에 게임 시작'은 코드에서 로비 상태 알리기 -> 게임 시작 신호 순서로 부르는 것으로 지킨다.
2. 입력 칸과 상태 칸은 쓰는 쪽도 읽는 쪽도 잠금 안에서 복사만 하고 바로 나온다. 쌓아 두지 않고 가장 최근 것 하나만 둔다.
3. 큐는 패킷 순서를 위해서가 아니라 스레드끼리 부딪히지 않게 하려고 쓴다. 메인 스레드는 큐를 빈 목록과 통째로 맞바꿔(swap) 꺼내고 바로 잠금을 푼다.
4. 서버의 모든 TCP 전송은 SendPacket(자리, 데이터, 길이) 하나로만 한다. 함수는 하나, 송신 잠금은 자리마다 하나다.

8. 상자 안을 채우는 구현 규칙

순서도 그림에는 없지만 그 상자를 코드로 옮길 때 꼭 지켜야 하는 규칙이다. 출처가 '제안'인 줄은 이 문서에서 처음 정한 것이라 팀이 바꿀 수 있다.

규칙	들어가는 상자	출처
클라의 UDP 소켓은 포트 0 으로 bind 한다(0 = 운영체제가 빈 포트를 골라 줌). bind 하지 않으면 recvfrom 이 WSAEINVAL 오류를 낸다.	접속 준비	제안

규칙	들어가는 상자	출처
'모두 준비했나?' 확인과 '게임 중' = true 로 바꾸기를 자리 잠금 안에서 한 번에 한다. 그래서 두 클라가 동시에 준비해도 바꾸는 데 성공한 스레드 하나만 SetEvent 를 부른다.	모두 준비 + 2 명 이상?, 게임 시작 신호	전체 순서도(판단과 시작 신호), 한 번에 하는 방법은 제안
자리 등록 때 그 자리의 마지막 입력 일련번호(lastSequence)를 0 으로 되돌린다.	자리 등록	제안
게임 준비 때 남아 있는 마지막 입력(이동, 발사)을 지운다.	게임 준비	제안
게임 준비 때 지금의 'E 누른 누적 횟수'(shieldPressCount)를 '지난 값'으로 기록해 둔다. 지난 판의 횟수 때문에 새 판 첫 틱에 쉴드가 저절로 켜지지 않게 하려는 것이다.	게임 준비	제안 (E 누른 횟수로 쉴드를 켜고 끄는 규칙에서 나옴)
UDP 주소를 아직 모르는 클라(첫 입력 패킷을 받기 전)에게는 상태를 보내지 않고 건너뛴다.	상태 보내기	제안
UDP 로 받은 패킷은 받은 바이트 수가 헤더의 size 와 같아야 한다. 입력 패킷은 playerId 가 0~3 이고 그 자리에 사람이 있어야 한다. 아니면 버린다.	입력 받기, UDP 받기	패킷 정의서(크기 검사), 제안(playerId 검사)
로비 상태는 어느 화면에서 받든 가장 최근 것으로 저장해 둔다.	큐 확인, 결과 화면, 로비 화면으로	제안
상태 패킷에 없는 적은 마지막 위치를 그대로 둔다. 적을 지우는 것은 사망 이벤트(SC_Event 의 EnemyDied)뿐이다. 이미 죽은 적의 위치가 오면 무시한다.	받은 것 반영	제안
서버의 TCP 전송은 SendPacket 하나로만 한다. 자리마다 송신 잠금(sendLock)이 있고, 그 안에서 패킷이 다 나갈 때까지 send 를 반복한다.	'(송신 임계 영역)' 표시가 있는 상자 전부	패킷 정의서
이벤트 큐에는 NetEvent(5-2)를 넣는다. 메인 스레드는 매 프레임 큐를 통째로 맞바꿔 꺼내고, 패킷 종류(switch)에 따라 처리한다.	TCP 받기, 응답 기다리기, 큐 확인, 받은 것 반영	전체 순서도(큐), 제안(NetEvent 와 맞바꿔 꺼내기)
거절 결과는 그 클라에게 자리가 없어서 SendPacket 을 쓸 수 없으므로 SendAll 로 바로 보낸다.	결과, 로비 상태 전송	제안
잠금 두 개를 겹쳐 잡지 않고 하나씩 차례로 잡는다(자리 잠금 -> 입력 잠금 -> 송신 잠금 순서). 겹쳐 잡으면 두 스레드가 서로 상대의 잠금을 기다리며 멈출 수 있다(데드락). 로비 상태는 자리 잠금 안에서 만들고, 잠금을 푼 뒤에 보낸다.	자리 등록, 로비 상태 알리기, 게임 준비, 로비로 되돌리기	제안

규칙	들어가는 상자	출처
소켓을 자리에 넣을 때는 그 자리의 송신 잠금 안에서 넣는다. 다른 스레드가 SendPacket 으로 같은 자리의 소켓을 읽기 때문이다.	자리 등록	제안
서버의 진행 단계는 '게임 중인가' bool 하나(m_inGame)로 나타내고, 패킷으로 보내지 않는다.	빈 자리 있고 로비 중인가?, 모두 준비 + 2 명 이상?	패킷 정의서(단계는 서버 안에만 둔다), 값을 bool 로 둔 것은 제안

9. 부록 1

항목	관련 순서도 상자	정한 것
자리 배열과 PlayerId	자리 등록	PlayerId = 자리 번호. 서버의 자리 배열 m_seats[4]에서 클라가 들어간 칸의 번호(0~3)를 그대로 PlayerId 로 쓴다. 빈 칸 중 가장 앞 칸을 주므로 접속한 순서대로 0, 1, 2, 3 이 된다. 테스트에서는 접속이 끊기는 경우를 다루지 않으므로 빈 칸을 다시 쓰는 일은 없다.
1 초에 30 번을 지키는 방법	틱 시작, 화면 그리기	클라 FrameAdvance 와 서버 게임 루프 모두 CGameTimer::Tick(30.0f)로 1/30 초를 맞춘다. Sleep 은 쓰지 않는다.
클라 표시 방식	게임 화면 만들기, 받은 것 반영	클라의 게임 화면(CGameScene)은 계산을 하지 않고, ApplyStates 로 서버가 보낸 값을 그대로 넣어 보여 준다(4-6).
서버 IP 입력	서버와 TCP 연결	클라 코드의 전역 상수 constexpr const char* SERVER_IP = "127.0.0.1"; 로 적는다(127.0.0.1 = 내 PC). 시연 때는 서버 PC 의 IP 로 바꿔서 빌드한다.
서버 종료 때 스레드 끝내기	(순서도에 없음. 서버 프로그램을 끌 때)	서버를 끌 때 만든 스레드와 소켓을 모두 정리하고 끝낸다. 자세한 순서는 나중에 정한다.

10. 부록 2 Protocol.h 전체

클라와 서버가 같이 쓰는 패킷 정의다. C++17 에서 컴파일해 크기를 확인했다.

```
#pragma once
#include <cstdint>

// Protocol.h : 클라이언트와 서버가 같이 쓰는 패킷 정의
// 모든 패킷은 PacketHeader(3 바이트)로 시작한다.

#pragma pack(push, 1)

// ===== 상수 =====
constexpr uint8_t MAX_PLAYERS = 4;              // 최대 접속 인원
constexpr uint8_t MIN_PLAYERS_TO_START = 2;     // 게임을 시작할 수 있는 최소 인원
constexpr uint8_t MAX_ENEMIES = 32;             // 한 맵의 AI 적 최대 수
constexpr uint8_t MAX_BULLETS = 64;             // 동시에 날아다니는 총알 최대 수
constexpr uint8_t INVALID_PLAYER_ID = 255;      // "없음" (거절됨, 승자 없음)
constexpr int TICK_RATE = 30;                   // 서버 틱, 클라이언트 프레임 (회/초)
constexpr uint16_t SERVER_TCP_PORT = 9000;      // 서버 TCP 대기 포트 (예시 값)
constexpr uint16_t SERVER_UDP_PORT = 9001;      // 서버 UDP 입력 받기 포트 (예시 값)

// ===== enum =====
enum class EPacketType : uint8_t
{
    // 클라이언트 -> 서버
    CS_Ready = 1,
    CS_Input,

    // 서버 -> 클라이언트
    SC_ConnectResult = 100,
    SC_LobbyState,
    SC_GameStart,
    SC_PlayerStates,
    SC_EnemyStates,
    SC_BulletStates,
    SC_Event,
    SC_GameResult
};

enum class EPlayerType : uint8_t { None = 0, Helicopter, Tank };            // 기체 종류
enum class EPlayerState : uint8_t { Alive = 0, Ghost };                      // Ghost = 사망 후 관전 (이동만 가능)
enum class EEventType : uint8_t { PlayerDied = 0, EnemyDied };              // SC_Event 종류
enum class ERejectReason : uint8_t { None = 0, ServerFull, AlreadyStarted }; // 접속 거절 이유

// ===== 공통 헤더 =====
struct PacketHeader
{
    uint16_t size;          // 헤더를 포함한 패킷 전체 바이트 수
    EPacketType type;       // 패킷 종류
};

// ===== 클라이언트 -> 서버 =====
struct CS_Ready             // TCP. 로비에서 대기 버튼을 처음 누를 때 1 회
{
    PacketHeader header;    // 내용 없음 (헤더만)
};

struct CS_Input             // UDP. 매 프레임 (30 회/초)
{
    PacketHeader header;
    uint8_t playerId;           // 보낸 플레이어 (UDP 는 연결이 없어서 패킷에 적음)
    uint32_t sequence;          // 입력 번호. 매 프레임 +1, 게임이 바뀌어도 되돌리지 않음
    int8_t moveForward;         // -1(S), 0, 1(W)
    int8_t moveRight;           // -1(A), 0, 1(D)
    float cameraYaw;            // 카메라 yaw (도). 이동 방향의 기준
    uint8_t fire;               // 좌클릭을 누르고 있으면 1
    uint8_t shieldPressCount;   // E 를 누른 횟수. 누를 때마다 +1, 서버는 값이 바뀌면 쉴드 토글
};

// ===== 서버 -> 클라이언트 =====
struct SC_ConnectResult     // TCP. accept 직후 그 클라이언트에게 1 회
```

```
{
    PacketHeader header;
    uint8_t accepted;           // 1 = 승인, 0 = 거절
    uint8_t playerId;           // 내 PlayerId (거절이면 INVALID_PLAYER_ID)
    uint8_t rejectReason;       // ERejectReason
};

struct LobbyPlayerInfo
{
    uint8_t playerId;
    uint8_t ready;              // 대기 버튼을 눌렀으면 1
};

struct SC_LobbyState        // TCP. 접속, 준비, 끊김, 로비 복귀 때 전원에게
{
    PacketHeader header;
    uint8_t playerCount;                        // 접속 인원 (앞에서 playerCount 칸이 유효)
    LobbyPlayerInfo players[MAX_PLAYERS];       // 항상 4 칸 모두 보냄
};

struct PlayerStartInfo
{
    uint8_t playerId;
    uint8_t playerType;         // EPlayerType
    float x, y, z;              // 시작 위치
};

struct SC_GameStart         // TCP. 게임 준비 때 전원에게 1 회
{
    PacketHeader header;
    uint8_t mapNumber;                          // 1 ~ 3
    uint8_t playerCount;                        // 이번 게임 참가 인원
    PlayerStartInfo players[MAX_PLAYERS];       // 항상 4 칸 모두 보냄
};

struct PlayerStateInfo
{
    uint8_t playerId;
    uint8_t state;              // EPlayerState (Alive / Ghost)
    uint8_t shieldEnabled;      // 쉴드가 켜져 있으면 1
    uint8_t health;             // 0 ~ 100
    float x, y, z;              // 위치
    float yaw;                  // 기체가 바라보는 방향 (도)
};

struct SC_PlayerStates      // UDP. 매 틱, 클라마다 sendto
{
    PacketHeader header;
    uint32_t tick;                              // 틱 번호. 게임이 바뀌어도 되돌리지 않음
    uint8_t count;                              // 들어 있는 플레이어 수 (나 포함, 최대 4)
    PlayerStateInfo players[MAX_PLAYERS];       // 플레이어마다 1 칸. count 칸만 보냄
};

struct EnemyStateInfo
{
    uint8_t enemyIndex;         // 게임 시작 때 맵 Spawn 순서 (서버와 클라가 같음, 종류도 이 번호로 앎)
    float x, y, z;              // 위치
    float yaw;                  // 바라보는 방향 (도)
};

struct SC_EnemyStates       // UDP. 매 틱, 클라마다 sendto
{
    PacketHeader header;
    uint32_t tick;
    uint8_t count;                              // 살아 있는 적 수
    EnemyStateInfo enemies[MAX_ENEMIES];        // 살아 있는 적만 count 칸 보냄 (죽음은 SC_Event 로 알림)
};

struct BulletStateInfo
{
    float x, y, z;              // 총알 위치
};

struct SC_BulletStates      // UDP. 매 틱, 클라마다 sendto
{
```

```
    PacketHeader header;
    uint32_t tick;
    uint8_t count;                              // 날아가는 총알 수
    BulletStateInfo bullets[MAX_BULLETS];       // count 칸만 보냄
};

struct SC_Event             // TCP. 사망이 일어난 틱에 전원에게
{
    PacketHeader header;
    uint8_t eventType;          // EEventType
    uint8_t subjectId;          // PlayerDied: 죽은 PlayerId / EnemyDied: enemyIndex
    float x, y, z;              // 사망 위치 (폭발 연출용)
};

struct SC_GameResult        // TCP. 생존자가 1 명 이하가 된 틱에 전원에게 1 회
{
    PacketHeader header;
    uint8_t winnerPlayerId;     // 승자. 동시 사망으로 승자가 없으면 INVALID_PLAYER_ID (전원 패배)
};

#pragma pack(pop)
```